

Язык программирования C++

Лекция 16: STL, стандартные контейнеры, итераторы,
инвалидация ссылок, инвалидация итераторов

Никита Лисица

2026

Напоминание: стандартная библиотека

- Поставляется вместе с реализацией (компилятором) языка

Напоминание: стандартная библиотека

- Поставляется вместе с реализацией (компилятором) языка
- Заголовочные файлы не имеют расширения:

```
#include <vector>  
#include <string>
```

Напоминание: стандартная библиотека

- Поставляется вместе с реализацией (компилятором) языка
- Заголовочные файлы не имеют расширения:

```
#include <vector>
#include <string>
```

- Большинство функций и классов лежат в `namespace std`:

```
std::vector<int> v = {1, 2, 3, 4};
std::string str = "I love Rust";
```

Напоминание: стандартная библиотека

- Поставляется вместе с реализацией (компилятором) языка
- Заголовочные файлы не имеют расширения:

```
#include <vector>
#include <string>
```

- Большинство функций и классов лежат в `namespace std`:

```
std::vector<int> v = {1, 2, 3, 4};
std::string str = "I love Rust";
```

- Широко использует всё, о чём мы говорили ранее в курсе: классы, шаблоны, исключения

- Standard Template Library (STL) – часть стандартной библиотеки, предоставляющая

- Standard Template Library (STL) – часть стандартной библиотеки, предоставляющая
 - Контейнеры (структуры данных для хранения и поиска)

- Standard Template Library (STL) – часть стандартной библиотеки, предоставляющая
 - Контейнеры (структуры данных для хранения и поиска)
 - Алгоритмы (для обработки данных)

- Standard Template Library (STL) – часть стандартной библиотеки, предоставляющая
 - Контейнеры (структуры данных для хранения и поиска)
 - Алгоритмы (для обработки данных)
- **NB:** часто термином STL называют всю стандартную библиотеку, что неверно

- Шаблоны классов, реализующие различные структуры данных

- Шаблоны классов, реализующие различные структуры данных
- Параметризуются типом хранимых объектов, от которых требуется
 - Иметь конструктор копирования (или move-конструктор)
 - Иметь оператор присваивания (или move-присваивания)
 - Иметь деструктор

- Все контейнеры имеют

- Все контейнеры имеют
 - Конструктор по умолчанию

- Все контейнеры имеют
 - Конструктор по умолчанию
 - Конструктор копирования и оператор присваивания

- Все контейнеры имеют
 - Конструктор по умолчанию
 - Конструктор копирования и оператор присваивания
 - Деструктор

- Все контейнеры имеют
 - Конструктор по умолчанию
 - Конструктор копирования и оператор присваивания
 - Деструктор
 - `size()` – размер контейнера

- Все контейнеры имеют
 - Конструктор по умолчанию
 - Конструктор копирования и оператор присваивания
 - Деструктор
 - `size()` – размер контейнера
 - `empty()` – проверить на пустоту

- Все контейнеры имеют
 - Конструктор по умолчанию
 - Конструктор копирования и оператор присваивания
 - Деструктор
 - `size()` – размер контейнера
 - `empty()` – проверить на пустоту
 - `clear()` – очистить (удалить все элементы)

- Все контейнеры имеют
 - Конструктор по умолчанию
 - Конструктор копирования и оператор присваивания
 - Деструктор
 - `size()` – размер контейнера
 - `empty()` – проверить на пустоту
 - `clear()` – очистить (удалить все элементы)
 - `swap(other)`

- Все контейнеры имеют
 - Конструктор по умолчанию
 - Конструктор копирования и оператор присваивания
 - Деструктор
 - `size()` – размер контейнера
 - `empty()` – проверить на пустоту
 - `clear()` – очистить (удалить все элементы)
 - `swap(other)`
 - `insert(...)`

- Все контейнеры имеют
 - Конструктор по умолчанию
 - Конструктор копирования и оператор присваивания
 - Деструктор
 - `size()` – размер контейнера
 - `empty()` – проверить на пустоту
 - `clear()` – очистить (удалить все элементы)
 - `swap(other)`
 - `insert(...)`
 - `erase(...)`

- Сохраняют порядок, в котором были добавлены элементы:

- Сохраняют порядок, в котором были добавлены элементы:
 - `std::vector` – саморасширяющийся массив

- Сохраняют порядок, в котором были добавлены элементы:
 - `std::vector` – саморасширяющийся массив
 - `std::list` – двусвязный список

- Сохраняют порядок, в котором были добавлены элементы:
 - `std::vector` – саморасширяющийся массив
 - `std::list` – двусвязный список
 - `std::deque` – дек (double-ended queue)

- Сохраняют порядок, в котором были добавлены элементы:
 - `std::vector` – саморасширяющийся массив
 - `std::list` – двусвязный список
 - `std::deque` – дек (double-ended queue)
 - `std::string` – строка (массив символов)

- Сохраняют порядок, в котором были добавлены элементы:
 - `std::vector` – саморасширяющийся массив
 - `std::list` – двусвязный список
 - `std::deque` – дек (double-ended queue)
 - `std::string` – строка (массив символов)
 - `std::array` – массив фиксированного размера (аналог `T[N]`)

- Сохраняют порядок, в котором были добавлены элементы:
 - `std::vector` – саморасширяющийся массив
 - `std::list` – двусвязный список
 - `std::deque` – дек (double-ended queue)
 - `std::string` – строка (массив символов)
 - `std::array` – массив фиксированного размера (аналог `T[N]`)
 - `std::forward_list` – односвязный список

- Саморасширяющийся массив: хранит элементы непрерывно в памяти, выделенной с запасом под новые элементы

- Саморасширяющийся массив: хранит элементы непрерывно в памяти, выделенной с запасом под новые элементы
- При исчерпании запаса, увеличивает размер (часто – в 2 раза) реаллоцируя память и копируя элементы

- Саморасширяющийся массив: хранит элементы непрерывно в памяти, выделенной с запасом под новые элементы
- При исчерпании запаса, увеличивает размер (часто – в 2 раза) реаллоцируя память и копируя элементы
- `size()` – количество элементов, `capacity()` – общий размер памяти (в элементах, не в байтах)

- Добавление/удаление в конец – амортизированное $O(1)$

- Добавление/удаление в конец – амортизированное $O(1)$
- Добавление/удаление в середине – $O(N)$, так как нужно сдвинуть остальные элементы

- Добавление/удаление в конец – амортизированное $O(1)$
- Добавление/удаление в середине – $O(N)$, так как нужно сдвинуть остальные элементы
- Доступ к элементу по индексу – $O(1)$

- Добавление/удаление в конец – амортизированное $O(1)$
- Добавление/удаление в середине – $O(N)$, так как нужно сдвинуть остальные элементы
- Доступ к элементу по индексу – $O(1)$
- Из-за кешей CPU, prefetching'a и т.п., **vector** – часто самый эффективный из контейнеров, и почти всегда это хороший дефолт

- `resize(n)` – изменить размер (количество реально хранимых элементов)

- `resize(n)` – изменить размер (количество реально хранимых элементов)
- `reserve(n)` – изменить размер памяти (запаса)

- `resize(n)` – изменить размер (количество реально хранимых элементов)
- `reserve(n)` – изменить размер памяти (запаса)
- `push_back(x)`/`pop_back()` – добавить/удалить последний элемент

- `resize(n)` – изменить размер (количество реально хранимых элементов)
- `reserve(n)` – изменить размер памяти (запаса)
- `push_back(x)`/`pop_back()` – добавить/удалить последний элемент
- `front()`/`back()` – получить ссылку на первый/последний элемент

- `resize(n)` – изменить размер (количество реально хранимых элементов)
- `reserve(n)` – изменить размер памяти (запаса)
- `push_back(x)`/`pop_back()` – добавить/удалить последний элемент
- `front()`/`back()` – получить ссылку на первый/последний элемент
- `operator[]` – доступ по индексу (выход за границу – UB)

- `resize(n)` – изменить размер (количество реально хранимых элементов)
- `reserve(n)` – изменить размер памяти (запаса)
- `push_back(x)`/`pop_back()` – добавить/удалить последний элемент
- `front()`/`back()` – получить ссылку на первый/последний элемент
- `operator[]` – доступ по индексу (выход за границу – UB)
- `at(i)` – доступ по индексу (выход за границу – исключение)

- `resize(n)` – изменить размер (количество реально хранимых элементов)
- `reserve(n)` – изменить размер памяти (запаса)
- `push_back(x)`/`pop_back()` – добавить/удалить последний элемент
- `front()`/`back()` – получить ссылку на первый/последний элемент
- `operator[]` – доступ по индексу (выход за границу – UB)
- `at(i)` – доступ по индексу (выход за границу – исключение)
- `data()` – получить указатель на хранимый массив (напр. для взаимодействия с библиотеками на C)

```
#include <vector>

std::vector<int> v(10);
v.push_back(42); // 11ый элемент
v[0] = 13;
v.at(5) = 14;
// v.at(11) -- бросит исключение
v.size(); // вернёт 11
v.pop_back();
v.empty(); // вернёт false
v.clear(); // после этого v.empty() == true

// Передаём данные в сишный API
glBufferData(v.data(), ...);
```

```
std::vector<int> v(10);

// Удалит все элементы кроме первых 5
v.resize(5);
// Добавит недостающие элементы конструктором по умолчанию
v.resize(20);
// Можно сказать, чем нужно заполнить новые элементы
v.resize(30, 5);

// Выделим память под 100 элементов (размер не изменится)
v.reserve(100);
// Удалим все элементы, но запас памяти останется под новые
v.clear();

// Загадка: что это и зачем?
std::vector<int>().swap(v);
```

- Double-ended queue: добавление/удаление с обоих концов за $O(1)$

- Double-ended queue: добавление/удаление с обоих концов за $O(1)$
- Обычно реализуется как массив указателей на массивы фиксированного размера (или даже вектор векторов)

- Double-ended queue: добавление/удаление с обоих концов за $O(1)$
- Обычно реализуется как массив указателей на массивы фиксированного размера (или даже вектор векторов)
- Гарантирует постоянство ссылок на элементы (иначе его можно было бы реализовать как циклический массив)

- Double-ended queue: добавление/удаление с обоих концов за $O(1)$
- Обычно реализуется как массив указателей на массивы фиксированного размера (или даже вектор векторов)
- Гарантирует постоянство ссылок на элементы (иначе его можно было бы реализовать как циклический массив)
- Обращение по индексу за $O(1)$

- `size()`/`resize(n)`

- `size()`/`resize(n)`
- `push_back(x)`/`pop_back()`

- `size()`/`resize(n)`
- `push_back(x)`/`pop_back()`
- `push_front(x)`/`pop_front()`

- `size()`/`resize(n)`
- `push_back(x)`/`pop_back()`
- `push_front(x)`/`pop_front()`
- `front()`/`back()`

- `size()`/`resize(n)`
- `push_back(x)`/`pop_back()`
- `push_front(x)`/`pop_front()`
- `front()`/`back()`
- `operator[]`/`at(i)`

- Обычный двусвязный список

- Обычный двусвязный список
- Для удобства реализации на концах всегда есть вспомогательные вершины, не хранящие данные

- Обычный двусвязный список
- Для удобства реализации на концах всегда есть вспомогательные вершины, не хранящие данные
- В качестве оптимизации эти вершины склеены в одну (т.е. список циклический) и хранятся полем в самом списке

- Обычный двусвязный список
- Для удобства реализации на концах всегда есть вспомогательные вершины, не хранящие данные
- В качестве оптимизации эти вершины склеены в одну (т.е. список циклический) и хранятся полем в самом списке
- Вставка/удаление в любом месте за $O(1)$

- Обычный двусвязный список
- Для удобства реализации на концах всегда есть вспомогательные вершины, не хранящие данные
- В качестве оптимизации эти вершины склеены в одну (т.е. список циклический) и хранятся полем в самом списке
- Вставка/удаление в любом месте за $O(1)$
- Нет обращения по индексу

- `size()/resize(n)`

- `size()`/`resize(n)`
- `push_back(x)`/`pop_back()`

- `size()`/`resize(n)`
- `push_back(x)`/`pop_back()`
- `push_front(x)`/`pop_front()`

- `size()`/`resize(n)`
- `push_back(x)`/`pop_back()`
- `push_front(x)`/`pop_front()`
- `merge(list)` – объединить два отсортированных списка в один

- `size()`/`resize(n)`
- `push_back(x)`/`pop_back()`
- `push_front(x)`/`pop_front()`
- `merge(list)` – объединить два отсортированных списка в один
- `splice(...)` – переместить часть одного списка в другой список

- По сути, `splice()` сводится к переподвешиванию указателей в вершинах списка, и должен выполняться за $O(1)$

Драма о `std::list::splice`

- По сути, `splice()` сводится к переподвешиванию указателей в вершинах списка, и должен выполняться за $O(1)$
- Проблема в том, как вычислять размер списка

Драма о `std::list::splice`

- По сути, `splice()` сводится к переподвешиванию указателей в вершинах списка, и должен выполняться за $O(1)$
- Проблема в том, как вычислять размер списка
- За $O(1)$ – хранить поле `size` и обновлять во всех операциях, тогда `splice()` придётся за $O(N)$ вычислять, сколько элементов было перенесено из одного списка в другой

Драма о `std::list::splice`

- По сути, `splice()` сводится к переподвешиванию указателей в вершинах списка, и должен выполняться за $O(1)$
- Проблема в том, как вычислять размер списка
- За $O(1)$ – хранить поле `size` и обновлять во всех операциях, тогда `splice()` придётся за $O(N)$ вычислять, сколько элементов было перенесено из одного списка в другой
- За $O(N)$ – вычислять проходом по списку, тогда `splice()` может работать за $O(1)$

Драма о `std::list::splice`

- По сути, `splice()` сводится к переподвешиванию указателей в вершинах списка, и должен выполняться за $O(1)$
- Проблема в том, как вычислять размер списка
- За $O(1)$ – хранить поле `size` и обновлять во всех операциях, тогда `splice()` придётся за $O(N)$ вычислять, сколько элементов было перенесено из одного списка в другой
- За $O(N)$ – вычислять проходом по списку, тогда `splice()` может работать за $O(1)$
- До C++11 не было специфицировано, какой именно вариант предоставляет стандартная библиотека

Драма о `std::list::splice`

- По сути, `splice()` сводится к переподвешиванию указателей в вершинах списка, и должен выполняться за $O(1)$
- Проблема в том, как вычислять размер списка
- За $O(1)$ – хранить поле `size` и обновлять во всех операциях, тогда `splice()` придётся за $O(N)$ вычислять, сколько элементов было перенесено из одного списка в другой
- За $O(N)$ – вычислять проходом по списку, тогда `splice()` может работать за $O(1)$
- До C++11 не было специфицировано, какой именно вариант предоставляет стандартная библиотека
- После C++11: `size()` за $O(1)$, `splice()` за $O(N)$

- Как `std::vector<char>`, но с особенностями специфичными для строк

- Как `std::vector<char>`, но с особенностями специфичными для строк
- Для совместимости с C в конце всегда хранит нулевой символ (не учитывается в `size()`)

- Как `std::vector<char>`, но с особенностями специфичными для строк
- Для совместимости с C в конце всегда хранит нулевой символ (не учитывается в `size()`)
- Есть неявный конструктор от строковых литералов
`string(const char *)`

- Как `std::vector<char>`, но с особенностями специфичными для строк
- Для совместимости с C в конце всегда хранит нулевой символ (не учитывается в `size()`)
- Есть неявный конструктор от строковых литералов `string(const char *)`
- Есть конкатенация строк через `operator+`

- Как `std::vector<char>`, но с особенностями специфичными для строк
- Для совместимости с C в конце всегда хранит нулевой символ (не учитывается в `size()`)
- Есть неявный конструктор от строковых литералов `string(const char *)`
- Есть конкатенация строк через `operator+`
- Множество специфичных для строк алгоритмов: `find()`, `substr()`, и т.п.

```
std::string str = "Hello";  
str += ", ";  
std::string str2 = "world!";  
std::string str3 = str + str2;  
  
if (str3.find(',') != std::string::npos) {  
    ...  
}
```

- Сам `std::string` – не шаблон, но специализация шаблона `std::basic_string<char>`

- Сам `std::string` – не шаблон, но специализация шаблона `std::basic_string<char>`
- `std::wstring = std::basic_string<wchar_t>` – строка *широких символов* (UTF-32 под Linux, UTF-16 под Windows, часто используются в WinAPI)

- Сам `std::string` – не шаблон, но специализация шаблона `std::basic_string<char>`
- `std::wstring` = `std::basic_string<wchar_t>` – строка *широких символов* (UTF-32 под Linux, UTF-16 под Windows, часто используются в WinAPI)
- `std::u8string` = `std::basic_string<char8_t>` – всегда UTF-8

- Сам `std::string` – не шаблон, но специализация шаблона `std::basic_string<char>`
- `std::wstring` = `std::basic_string<wchar_t>` – строка *широких символов* (UTF-32 под Linux, UTF-16 под Windows, часто используются в WinAPI)
- `std::u8string` = `std::basic_string<char8_t>` – всегда UTF-8
- `std::u16string` = `std::basic_string<char16_t>` – всегда UTF-16

- Сам `std::string` – не шаблон, но специализация шаблона `std::basic_string<char>`
- `std::wstring` = `std::basic_string<wchar_t>` – строка *широких символов* (UTF-32 под Linux, UTF-16 под Windows, часто используются в WinAPI)
- `std::u8string` = `std::basic_string<char8_t>` – всегда UTF-8
- `std::u16string` = `std::basic_string<char16_t>` – всегда UTF-16
- `std::u32string` = `std::basic_string<char32_t>` – всегда UTF-32

- Реализуют интерфейс некоторого контейнера

Адаптеры контейнеров

- Реализуют интерфейс некоторого контейнера
- Параметризуются другим контейнером, используемым для хранения элементов

Адаптеры контейнеров

- Реализуют интерфейс некоторого контейнера
- Параметризуются другим контейнером, используемым для хранения элементов
- `std::stack<int, std::vector<int>>` – стек (есть `push()`/`pop()`), использующий вектор для хранения

Адаптеры контейнеров

- Реализуют интерфейс некоторого контейнера
- Параметризуются другим контейнером, используемым для хранения элементов
- `std::stack<int, std::vector<int>>` – стек (есть `push()`/`pop()`), использующий вектор для хранения
- `std::queue<int, std::list<int>>` – очередь (есть `push()`/`pop()`), использующая список для хранения

Адаптеры контейнеров

- Реализуют интерфейс некоторого контейнера
- Параметризуются другим контейнером, используемым для хранения элементов
- `std::stack<int, std::vector<int>>` – стек (есть `push()`/`pop()`), использующий вектор для хранения
- `std::queue<int, std::list<int>>` – очередь (есть `push()`/`pop()`), использующая список для хранения
- `std::priority_queue<int, std::deque<int>>` – приоритетная очередь (куча), использующая деку для хранения

Адаптеры контейнеров

- Реализуют интерфейс некоторого контейнера
- Параметризуются другим контейнером, используемым для хранения элементов
- `std::stack<int, std::vector<int>>` – стек (есть `push()`/`pop()`), использующий вектор для хранения
- `std::queue<int, std::list<int>>` – очередь (есть `push()`/`pop()`), использующая список для хранения
- `std::priority_queue<int, std::deque<int>>` – приоритетная очередь (куча), использующая деку для хранения
- **NB:** для работы с приоритетной очередью часто удобнее использовать свободные функции `std::make_heap`/`push_heap`/`pop_heap`

- В общем случае – объект, позволяющий итерироваться по последовательности элементов некоторой коллекции/контейнера

- В общем случае – объект, позволяющий итерироваться по последовательности элементов некоторой коллекции/контейнера
- Есть два стиля реализации итераторов – условно, Java и C++

- Java-style: итератор позволяет один раз пройти по коллекции

- Java-style: итератор позволяет один раз пройти по коллекции

```
interface Iterator<T> {  
    // Достигнут ли конец последовательности  
    bool hasNext();  
    // Возвращает следующий элемент последовательности  
    T next();  
}  
  
while (it.hasNext()) {  
    T value = it.next();  
    ...  
}
```

Итераторы: C++ style

- C++-style: итератор указывает на конкретный элемент, и позволяет итерироваться в разные стороны

Итераторы: C++ style

- C++-style: итератор указывает на конкретный элемент, и позволяет итерироваться в разные стороны
- Сам итератор не знает, где конец последовательности, нужны два итератора `begin/end` – *iterator range*

Итераторы: C++ style

- C++-style: итератор указывает на конкретный элемент, и позволяет итерироваться в разные стороны
- Сам итератор не знает, где конец последовательности, нужны два итератора `begin/end` – *iterator range*
- Имеют интерфейс, аналогичный указателям

Итераторы: C++ style

- C++-style: итератор указывает на конкретный элемент, и позволяет итерироваться в разные стороны
- Сам итератор не знает, где конец последовательности, нужны два итератора `begin/end` – *iterator range*
- Имеют интерфейс, аналогичный указателям

```
class iterator {  
public:  
    // Возвращает текущий элемент последовательности  
    T& operator*();  
  
    // Переходит к следующему элементу последовательности  
    iterator& operator++();  
};  
  
for (iterator it = begin; it != end; ++it) {  
    T& value = *it;  
    ...  
}
```

- Все STL-контейнеры работают с итераторами (обычно это вложенные классы, напр. `std::vector<int>::iterator`)

- Все STL-контейнеры работают с итераторами (обычно это вложенные классы, напр. `std::vector<int>::iterator`)
- Релизация итератора зависит от контейнера, но почти всегда это просто указатель на элемент / вершину списка / etc

- Все STL-контейнеры работают с итераторами (обычно это вложенные классы, напр. `std::vector<int>::iterator`)
- Релизация итератора зависит от контейнера, но почти всегда это просто указатель на элемент / вершину списка / etc
- `begin()/end()` – получить итератор на начало/конец

- Все STL-контейнеры работают с итераторами (обычно это вложенные классы, напр. `std::vector<int>::iterator`)
- Релизация итератора зависит от контейнера, но почти всегда это просто указатель на элемент / вершину списка / etc
- `begin()/end()` – получить итератор на начало/конец
- `find(x)` – возвращает итератор на найденный элемент (или на конец)

- Все STL-контейнеры работают с итераторами (обычно это вложенные классы, напр. `std::vector<int>::iterator`)
- Релизация итератора зависит от контейнера, но почти всегда это просто указатель на элемент / вершину списка / etc
- `begin()/end()` – получить итератор на начало/конец
- `find(x)` – возвращает итератор на найденный элемент (или на конец)
- `erase(it)` – удаляет элемент, на который указывает итератор

- Все STL-контейнеры работают с итераторами (обычно это вложенные классы, напр. `std::vector<int>::iterator`)
- Релизация итератора зависит от контейнера, но почти всегда это просто указатель на элемент / вершину списка / etc
- `begin()/end()` – получить итератор на начало/конец
- `find(x)` – возвращает итератор на найденный элемент (или на конец)
- `erase(it)` – удаляет элемент, на который указывает итератор
- `insert(it, x)` – вставляет элемент *перед* тем, на который указывает итератор

- 'Итератор на конец' почти всегда указывает не на последний элемент, а за последний элемент

- ‘Итератор на конец’ почти всегда указывает не на последний элемент, а за последний элемент
- Для `std::vector` – на элемент `v.data() + v.size()`

- ‘Итератор на конец’ почти всегда указывает не на последний элемент, а за последний элемент
- Для `std::vector` – на элемент `v.data() + v.size()`
- Для `std::list` – на фиктивную вершину

- ‘Итератор на конец’ почти всегда указывает не на последний элемент, а за последний элемент
- Для `std::vector` – на элемент `v.data() + v.size()`
- Для `std::list` – на фиктивную вершину
- Это оказывается гораздо удобнее для многих алгоритмов

End итератор

- ‘Итератор на конец’ почти всегда указывает не на последний элемент, а за последний элемент
- Для `std::vector` – на элемент `v.data() + v.size()`
- Для `std::list` – на фиктивную вершину
- Это оказывается гораздо удобнее для многих алгоритмов
- Например, если диапазон `[begin, end)` разбить на два некоторым элементом `middle`, то получатся два диапазона: `[begin, middle)` и `[middle, end)`

- В чём проблема такого кода?

```
std::vector<int> v(10);  
int& x = v[5];  
v.push_back(1);  
std::cout << x << std::endl;
```

- В чём проблема такого кода?

```
std::vector<int> v(10);  
int& x = v[5];  
v.push_back(1);  
std::cout << x << std::endl;
```

- `push_back()` может перевыделить память $\implies$ ссылка `x` может указывать на уже освобождённую память

- В чём проблема такого кода?

```
std::vector<int> v(10);  
int& x = v[5];  
v.push_back(1);  
std::cout << x << std::endl;
```

- `push_back()` может перевыделить память $\implies$ ссылка `x` может указывать на уже освобождённую память
- Говорят, что ссылка *инвалидировалась/инвалидирована*

Инвалидация итераторов

- В чём проблема такого кода?

```
std::vector<int> v(10);  
// Итераторы вектора умеют прибавлять/вычитать  
// числа, как указатели  
auto it = v.begin() + 5;  
v.push_back(1);  
std::cout << *it << std::endl;
```

Инвалидация итераторов

- В чём проблема такого кода?

```
std::vector<int> v(10);  
// Итераторы вектора умеют прибавлять/вычитать  
// числа, как указатели  
auto it = v.begin() + 5;  
v.push_back(1);  
std::cout << *it << std::endl;
```

- `push_back()` может перевыделить память $\implies$ итератор `it` может указывать на уже освобождённую память

Инвалидация итераторов

- В чём проблема такого кода?

```
std::vector<int> v(10);  
// Итераторы вектора умеют прибавлять/вычитать  
// числа, как указатели  
auto it = v.begin() + 5;  
v.push_back(1);  
std::cout << *it << std::endl;
```

- `push_back()` может перевыделить память $\implies$ итератор `it` может указывать на уже освобождённую память
- Говорят, что итератор *инвалидировался/инвалидирован*

- Когда происходит инвалидация ссылок/итераторов?

Инвалидация итераторов

- Когда происходит инвалидация ссылок/итераторов?
- Можно прочитать в документации

Инвалидация итераторов

- Когда происходит инвалидация ссылок/итераторов?
- Можно прочитать в документации, но проще знать, как работают контейнеры :)

Инвалидация итераторов

- Когда происходит инвалидация ссылок/итераторов?
- Можно прочитать в документации, но проще знать, как работают контейнеры :)
- `std::vector` – когда может перевыделиться память (при добавлении элемента), при удалении элемента (инвалидируются только итераторы *после* удалённого элемента)

Инвалидация итераторов

- Когда происходит инвалидация ссылок/итераторов?
- Можно прочитать в документации, но проще знать, как работают контейнеры :)
- `std::vector` – когда может перевыделиться память (при добавлении элемента), при удалении элемента (инвалидируются только итераторы *после* удалённого элемента)
- `std::deque` – добавление/удаление инвалидирует итераторы (перевыделяется массив указателей на блоки), но не ссылки на элементы

Инвалидация итераторов

- Когда происходит инвалидация ссылок/итераторов?
- Можно прочесть в документации, но проще знать, как работают контейнеры :)
- `std::vector` – когда может перевыделиться память (при добавлении элемента), при удалении элемента (инвалидируются только итераторы *после* удалённого элемента)
- `std::deque` – добавление/удаление инвалидирует итераторы (перевыделяется массив указателей на блоки), но не ссылки на элементы
- `std::list` – только если удаляется конкретный элемент (инвалидируется итератор на него, и только он)

Инвалидация итераторов

- В чём проблема такого кода?

```
// Удаляем все чётные элементы
for (auto it = v.begin(); it != v.end(); ++it)
    if (*it % 2 == 0)
        v.erase(it);
```

Инвалидация итераторов

- В чём проблема такого кода?

```
// Удаляем все чётные элементы
for (auto it = v.begin(); it != v.end(); ++it)
    if (*it % 2 == 0)
        v.erase(it);
```

- `erase()` может инвалидировать итератор

- `erase(it)` удаляет элемент, и возвращает итератор на следующий

Инвалидация итераторов

- `erase(it)` удаляет элемент, и возвращает итератор на следующий
- В чём проблема такого кода?

```
// Удаляем все чётные элементы
for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it % 2 == 0)
        it = v.erase(it);
}
```

Инвалидация итераторов

- `erase(it)` удаляет элемент, и возвращает итератор на следующий
- В чём проблема такого кода?

```
// Удаляем все чётные элементы
for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it % 2 == 0)
        it = v.erase(it);
}
```

- Пропустим все элементы, следующие за чётными

Инваляция итераторов

```
// Удаляем все чётные элементы
for (auto it = v.begin(); it != v.end();) {
    if (*it % 2 == 0)
        it = v.erase(it);
    else
        ++it;
}
```

Инвалидация итераторов

- В чём проблема такого кода?

```
// Вставляем 0 перед каждым чётным элементом
for (auto it = v.begin(); it != v.end(); ++it)
    if (*it % 2 == 0)
        v.insert(it, 0);
```

Инвалидация итераторов

- В чём проблема такого кода?

```
// Вставляем 0 перед каждым чётным элементом
for (auto it = v.begin(); it != v.end(); ++it)
    if (*it % 2 == 0)
        v.insert(it, 0);
```

- `insert()` может инвалидировать итератор

- `insert(it, value)` вставляет элемент, и возвращает новый итератор на него

Инвалидация итераторов

- `insert(it, value)` вставляет элемент, и возвращает новый итератор на него
- В чём проблема такого кода?

```
// Вставляем 0 перед каждым чётным элементом
for (auto it = v.begin(); it != v.end(); ++it)
    if (*it % 2 == 0)
        it = v.insert(it, 0);
```

Инвалидация итераторов

- `insert(it, value)` вставляет элемент, и возвращает новый итератор на него
- В чём проблема такого кода?

```
// Вставляем 0 перед каждым чётным элементом
for (auto it = v.begin(); it != v.end(); ++it)
    if (*it % 2 == 0)
        it = v.insert(it, 0);
```

- Бесконечно будем вставлять 0 перед одним и тем же элементом

Инваляция итераторов

```
// Вставляем 0 перед каждым чётным элементом
for (auto it = v.begin(); it != v.end(); ++it) {
    if (*it % 2 == 0) {
        it = v.insert(it, 0);
        ++it;
    }
}
```

- Хранят элементы по-особому для быстрого поиска

- Хранят элементы по-особому для быстрого поиска
- Упорядоченные:
 - `set`, `multiset`, `map`, `multimap`

- Хранят элементы по-особому для быстрого поиска
- Упорядоченные:
 - `set`, `multiset`, `map`, `multimap`
 - Реализуются в виде дерева (обычно – красно-чёрное)

Ассоциативные контейнеры

- Хранят элементы по-особому для быстрого поиска
- Упорядоченные:
 - `set, multiset, map, multimap`
 - Реализуются в виде дерева (обычно – красно-чёрное)
 - Требуют отношение порядка на элементах (`operator <`)

Ассоциативные контейнеры

- Хранят элементы по-особому для быстрого поиска
- Упорядоченные:
 - `set`, `multiset`, `map`, `multimap`
 - Реализуются в виде дерева (обычно – красно-чёрное)
 - Требуют отношение порядка на элементах (`operator <`)
 - Операции выполняются за $O(\log N)$

Ассоциативные контейнеры

- Хранят элементы по-особому для быстрого поиска
- Упорядоченные:
 - `set`, `multiset`, `map`, `multimap`
 - Реализуются в виде дерева (обычно – красно-чёрное)
 - Требуют отношение порядка на элементах (`operator <`)
 - Операции выполняются за $O(\log N)$
- Непорядоченные:
 - `unordered_set`, `unordered_multiset`, `unordered_map`, `unordered_multimap`

Ассоциативные контейнеры

- Хранят элементы по-особому для быстрого поиска
- Упорядоченные:
 - `set`, `multiset`, `map`, `multimap`
 - Реализуются в виде дерева (обычно – красно-чёрное)
 - Требуют отношение порядка на элементах (`operator <`)
 - Операции выполняются за $O(\log N)$
- Непорядоченные:
 - `unordered_set`, `unordered_multiset`, `unordered_map`, `unordered_multimap`
 - Реализуются в виде хеш-таблицы с закрытой адресацией

Ассоциативные контейнеры

- Хранят элементы по-особому для быстрого поиска
- Упорядоченные:
 - `set`, `multiset`, `map`, `multimap`
 - Реализуются в виде дерева (обычно – красно-чёрное)
 - Требуют отношение порядка на элементах (**operator <**)
 - Операции выполняются за $O(\log N)$
- Непорядоченные:
 - `unordered_set`, `unordered_multiset`, `unordered_map`, `unordered_multimap`
 - Реализуются в виде хеш-таблицы с закрытой адресацией
 - Требуют функцию, вычисляющую хеш

Ассоциативные контейнеры

- Хранят элементы по-особому для быстрого поиска
- Упорядоченные:
 - `set`, `multiset`, `map`, `multimap`
 - Реализуются в виде дерева (обычно – красно-чёрное)
 - Требуют отношение порядка на элементах (**operator <**)
 - Операции выполняются за $O(\log N)$
- Непорядоченные:
 - `unordered_set`, `unordered_multiset`, `unordered_map`, `unordered_multimap`
 - Реализуются в виде хеш-таблицы с закрытой адресацией
 - Требуют функцию, вычисляющую хеш
 - Операции выполняются за $O(1)$ в среднем, за $O(N)$ в худшем случае

- `set`, `unordered_set` – хранят уникальные элементы (без повторений)

- `set`, `unordered_set` – хранят уникальные элементы (без повторений)
- `multiset`, `unordered_multiset` – элементы могут повторяться

- `set`, `unordered_set` – хранят уникальные элементы (без повторений)
- `multiset`, `unordered_multiset` – элементы могут повторяться
- `map`, `unordered_map` – хранят пары (ключ, значение) для уникальных ключей

- `set`, `unordered_set` – хранят уникальные элементы (без повторений)
- `multiset`, `unordered_multiset` – элементы могут повторяться
- `map`, `unordered_map` – хранят пары (ключ, значение) для уникальных ключей
- `multimap`, `unordered_multimap` – ключи могут повторяться

```
std::set<int> set;
set.insert(2);
set.insert(1);
set.insert(1); // вторая копия НЕ добавится
set.insert(9);
set.insert(5);

set.contains(1); // true
set.contains(3); // false
set.size(); // 4

// Выведется 1 2 5 9
for (auto it = set.begin(); it != set.end(); ++it) {
    std::cout << *it << " ";
}
```

```
std::multiset<int> set;
set.insert(2);
set.insert(1);
set.insert(1); // вторая копия добавится
set.insert(9);
set.insert(5);

set.contains(1); // true
set.contains(3); // false
set.size(); // 5

// Выведется 1 1 2 5 9
for (auto it = set.begin(); it != set.end(); ++it) {
    std::cout << *it << " ";
}
```


unordered_set

```
std::unordered<int> set;
set.insert(2);
set.insert(1);
set.insert(1); // вторая копия НЕ добавится
set.insert(9);
set.insert(5);

set.contains(1); // true
set.contains(3); // false
set.size(); // 4

// Выведутся элементы в каком-то порядке,
// например 5 2 1 9
for (auto it = set.begin(); it != set.end(); ++it) {
    std::cout << *it << " ";
}
```

- У `*set` итераторы *константные*, т.е. не позволяют модифицировать элемент, на который ссылаются

Константные итераторы

- У `*set` итераторы *константные*, т.е. не позволяют модифицировать элемент, на который ссылаются
- Иначе нарушились бы внутренние инварианты структуры данных (порядок в дереве, индекс в хеш-таблице)

Константные итераторы

- У `*set` итераторы *константные*, т.е. не позволяют модифицировать элемент, на который ссылаются
- Иначе нарушились бы внутренние инварианты структуры данных (порядок в дереве, индекс в хеш-таблице)
- У всех контейнеров есть отдельные типы `iterator` и `const_iterator` (у `*set` они совпадают)

Константные итераторы

- У `*set` итераторы *константные*, т.е. не позволяют модифицировать элемент, на который ссылаются
- Иначе нарушились бы внутренние инварианты структуры данных (порядок в дереве, индекс в хеш-таблице)
- У всех контейнеров есть отдельные типы `iterator` и `const_iterator` (у `*set` они совпадают)
- `cbegin()/cend()` возвращают константные итераторы

- `std::pair<X, Y>` – вспомогательный класс, хранящий пару элементов

- `std::pair<X, Y>` – вспомогательный класс, хранящий пару элементов

```
template <typename X, typename Y>
struct pair {
    X first;
    Y second;
};
```

- `std::pair<X, Y>` – вспомогательный класс, хранящий пару элементов

```
template <typename X, typename Y>
struct pair {
    X first;
    Y second;
};
```

- Различные варианты контейнера `map<Key, Value>` хранят в себе `std::pair<const Key, Value>`

- `std::pair<X, Y>` – вспомогательный класс, хранящий пару элементов

```
template <typename X, typename Y>
struct pair {
    X first;
    Y second;
};
```

- Различные варианты контейнера `map<Key, Value>` хранят в себе `std::pair<const Key, Value>`
- В остальном `map` аналогичен `set`

map, std::pair

```
std::map<std::string, int> settings;

// Явное создание шаблона std::pair
settings.insert(std::pair<std::string, int>("volume", 100));

// Использование вспомогательной функции
settings.insert(std::make_pair("brightness", 80));

// Использование вывода параметров шаблонов (C++17)
settings.insert(std::pair("ui_scale", 3));

// Использование list initialization
settings.insert({"difficulty", 5});

// Итерация по элементам:
for (auto it = settings.begin(); it != settings.end(); ++it)
    std::cout << it->first << ": " << it->second << std::endl;
```

- У map и unordered_map есть **operator[]**, упрощающий работу с ними:

```
std::map<std::string, int> settings;  
settings["volume"] = 100;  
settings["brightness"] = 80;  
  
// Что произойдёт здесь?  
std::cout << settings["ui_scale"];
```

map: operator[]

- У `map` и `unordered_map` есть `operator[]`, упрощающий работу с ними:

```
std::map<std::string, int> settings;  
settings["volume"] = 100;  
settings["brightness"] = 80;  
  
// Что произойдёт здесь?  
std::cout << settings["ui_scale"];
```

- Обращение к несуществующему элементу через `operator[]` создаст его с помощью конструктора по умолчанию

map: operator[]

- У map и unordered_map есть `operator[]`, упрощающий работу с ними:

```
std::map<std::string, int> settings;  
settings["volume"] = 100;  
settings["brightness"] = 80;  
  
// Что произойдёт здесь?  
std::cout << settings["ui_scale"];
```

- Обращение к несуществующему элементу через `operator[]` создаст его с помощью конструктора по умолчанию
- $\implies$ Нужно использовать аккуратно, либо заменить на `map::find()`